Informe de Desarrollo

Desafío II – UdeAWorldCup

Informática II

Integrante 1: *Jose Manuel Posada Londoño*

Integrante 2: *Camilo Andres Villanueva Gonzalez*

Universidad de Antioquia

2026

Repositorio: https://github.com/josemanuelposada-collab/Desafio-2---Informatica-2---Jose-Manuel-Posada-Londo-o---Camilo-Andres-Villanueva-Gonza git

Video: *https://www.youtube.com/watch?v=28cH9jvoiWQ*

1. Introducción

En este informe se presenta el proceso de análisis, diseño e implementación del proyecto UdeAWorldCup como una simulación del Mundial de Fútbol 2026 usando programación orientada a objetos.

El principal objetivo del proyecto fue representar diferentes elementos de un torneo de fútbol, como jugadores, equipos, grupos, partidos y fases de eliminación, utilizando estructuras dinámicas de datos, modularidad mediante archivos `.h` y `.cpp`, y criterios en la eficiencia de la memoria.

La solución permite cargar la información de 48 selecciones desde un archivo CSV, crear plantillas artificiales de jugadores, organizar bombos, sortear grupos, simular la fase de grupos, clasificar equipos a dieciseisavos y continuar con las fases finales hasta obtener un campeón. Además, se incluyó un medidor de recursos para estimar iteraciones y consumo de memoria en cada funcionalidad importante.

2. Análisis del problema

El problema consiste en modelar y simular un torneo mundial de fútbol con 48 equipos. Cada selección contiene información histórica, pertenece a una confederación, tiene un ranking FIFA, un director técnico y una plantilla de jugadores.

El torneo se divide en dos etapas principales: la fase de grupos, compuesta por 12 grupos de 4 equipos, y las fases finales, que incluyen dieciseisavos, octavos, cuartos, semifinales, tercer puesto y final.

Desde el punto de vista de programación, el reto exige representar relaciones entre varias entidades. Un equipo contiene jugadores, un grupo contiene referencias a equipos, un partido relaciona dos selecciones y el torneo administra todas las estructuras principales.

Se trabajó con arreglos dinámicos, punteros y cadenas dinámicas de caracteres `char*`. Esto permitió un mayor control sobre la memoria, aunque también obligó a implementar constructores de copia, operadores de asignación y destructores en las clases que administran recursos dinámicos.

3. Diseño de clases

Las clases principales implementadas fueron:

- Jugador
- Equipo
- Grupo

- Fecha
- Partido
- RegistroJugadorPartido
- Torneo
- MedidorRecursos

4. Descripción de clases

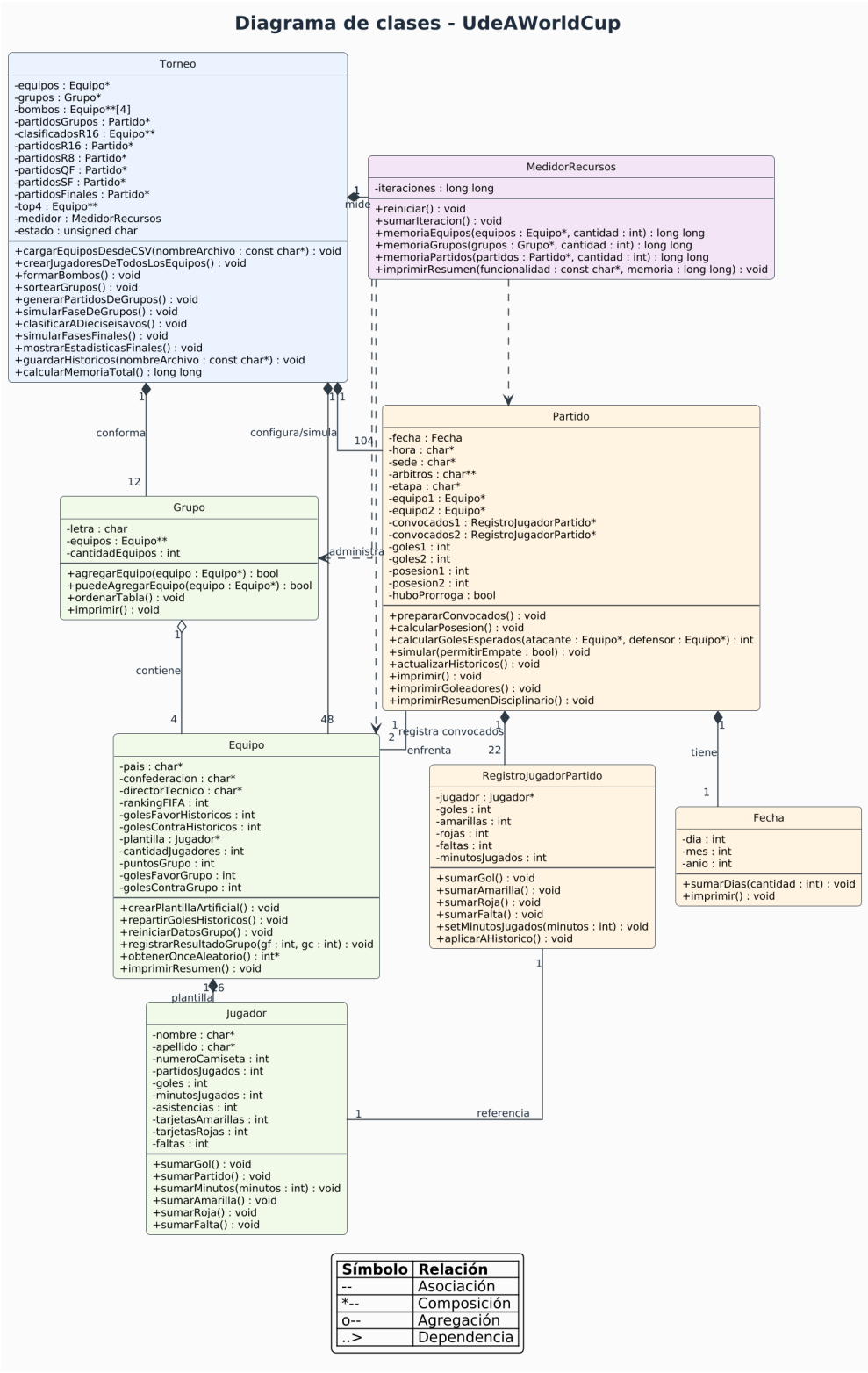

Figura 1: Diagrama de clases de la solución UdeAWorldCup.

4.1. Clase Jugador

La clase Jugador representa a cada jugador de una selección. Contiene información personal básica y estadísticas acumuladas.

Entre sus atributos se encuentran:

- `char* nombre`
- `char* apellido`
- `int numeroCamiseta`
- `int partidosJugados`
- `int goles`
- `int minutosJugados`
- `int asistencias`
- `int tarjetasAmarillas`
- `int tarjetasRojas`
- `int faltas`

Como esta clase reserva memoria dinámica para el nombre y el apellido, implementa constructor, constructor de copia, operador de asignación y destructor.

4.2. Clase Equipo

La clase Equipo representa una selección nacional. Contiene datos como país, confederación, director técnico, ranking FIFA y estadísticas históricas.

También contiene un arreglo dinámico de jugadores:

```
Jugador* plantilla;
```

La plantilla se crea dinámicamente con 26 jugadores mediante la función `crearPlantillaArtificial()`. Los goles históricos del equipo se reparten entre sus jugadores para dar coherencia inicial a las estadísticas.

Esta clase es una de las más importantes del sistema porque centraliza las estadísticas acumuladas que se actualizan después de cada partido.

4.3. Clase Grupo

La clase Grupo representa un grupo del Mundial. Contiene una letra identificadora y un arreglo dinámico de punteros a equipos:

```
Equipo** equipos;
```

El grupo no es dueño de los equipos; solo almacena referencias a los objetos creados por la clase Torneo. Por esa razón, su destructor libera el arreglo de punteros, pero no destruye los objetos `Equipo`.

La función `puedeAgregarEquipo()` verifica las restricciones de confederación. No permite dos equipos de la misma confederación en el mismo grupo, salvo UEFA, que puede aparecer como máximo dos veces.

4.4. Clase Fecha

La clase Fecha permite representar fechas de partidos. Contiene día, mes y año. Sus funciones permiten sumar días, comparar fechas e imprimirlas en pantalla.

Esta clase se usó para asignar fechas a los partidos de fase de grupos y de fases finales.

4.5. Clase RegistroJugadorPartido

Esta clase representa las estadísticas de un jugador en un partido específico. Contiene un puntero al jugador original y valores como goles, tarjetas, faltas y minutos jugados.

Su función `aplicarAHistorico()` actualiza las estadísticas acumuladas del jugador al finalizar el partido.

4.6. Clase Partido

La clase Partido representa un encuentro entre dos equipos. Contiene:

- fecha
- hora
- sede
- árbitros
- etapa
- equipo 1
- equipo 2

- convocados de cada equipo
- goles
- posesión
- indicador de prórroga

La clase Partido no es dueña de los equipos, por lo que no libera `equipo1` ni `equipo2`. Solo libera sus propios recursos, como cadenas dinámicas, árbitros y arreglos de convocados.

4.7. Clase Torneo

La clase Torneo administra el flujo general del programa. Es responsable de:

- cargar los equipos desde CSV;
- crear jugadores;
- formar bombos;
- sortear grupos;
- generar partidos de grupos;
- simular la fase de grupos;
- clasificar a dieciseisavos;
- simular fases finales;
- mostrar estadísticas finales;
- medir recursos.

Entre sus estructuras dinámicas principales están:

```
Equipo* equipos;
Grupo* grupos;
Partido* partidosGrupos;
Equipo** clasificadosR16;
Partido* partidosR16;
Partido* partidosR8;
Partido* partidosQF;
Partido* partidosSF;
Partido* partidosFinales;
Equipo** top4;
```

4.8. Clase MedidorRecursos

La clase MedidorRecursos estima la cantidad de iteraciones y la memoria consumida por el sistema como una estimación basada en las estructuras creadas dinámicamente.

5. Algoritmos principales

5.1. Carga de equipos desde CSV

La carga de equipos se realiza leyendo línea por línea el archivo `selecciones_clasificadas_mundial.c` El archivo usa punto y coma como separador, por lo que se implementó una forma manual de leer y separar los datos sin librerías externas.

El algoritmo general es el siguiente:

1. Abrir el archivo CSV.
2. Leer línea por línea.
3. Dividir cada línea usando el carácter `;` como separador.
4. Ignorar encabezados mediante la función `esNumeroCSV()`.
5. Crear objetos `Equipo` con los datos leídos.
6. Guardar los equipos en un arreglo dinámico.

5.2. Creación artificial de jugadores

Cada equipo crea una plantilla de 26 jugadores. Los nombres y apellidos se generan de forma artificial usando etiquetas como:

```
nombre1, apellido1
nombre2, apellido2
...
nombre26, apellido26
```

Además, los goles históricos del equipo se reparten entre sus jugadores para que las estadísticas iniciales tengan cierta coherencia con los datos históricos.

5.3. Formación de bombos

Para formar los bombos, primero se ordenan los equipos por ranking FIFA usando un algoritmo de burbuja. Luego se reparten en cuatro bombos de 12 equipos.

5.4. Sorteo de grupos

El sorteo de grupos toma un equipo de cada bombo e intenta ubicarlo en un grupo cumpliendo restricciones de confederación. La condición aplicada fue:

- no se permite repetir confederación en un grupo;
- UEFA puede aparecer máximo dos veces por grupo.

5.5. Generación de partidos de grupos

Cada grupo tiene 4 equipos, por lo tanto se generan 6 partidos por grupo. Como hay 12 grupos, se generan 72 partidos en total.

La combinación usada para los equipos dentro de cada grupo garantiza que todos jueguen contra todos una vez.

5.6. Simulación de goles

Para simular los goles esperados de un equipo contra otro, se usa la ecuación indicada en el enunciado:

$$\lambda_A = \mu \left(\frac{GFA}{\mu}\right)^{\alpha} \left(\frac{GCB}{\mu}\right)^{\beta}$$

Donde:

- GFA es el promedio de goles a favor del equipo A.
- GCB es el promedio de goles en contra del equipo B.
- $\alpha = 0{,}6$
- $\beta = 0{,}4$
- $\mu = 1{,}35$

La función se calcula para cada equipo según su rival. Es decir, para un partido entre A y B se calcula:

$$\lambda_A = f(A, B), \qquad \lambda_B = f(B, A)$$

Esto hace que cada resultado dependa del ataque histórico de un equipo y de la defensa histórica del rival.

5.7. Convocados y estadísticas por jugador

En cada partido se seleccionan aleatoriamente 11 jugadores de cada equipo. Para cada jugador convocado se registran:

- goles;
- tarjetas amarillas;
- tarjetas rojas;
- faltas;
- minutos jugados.

Al finalizar el partido, las estadísticas del registro se aplican al histórico de cada jugador.

5.8. Clasificación a dieciseisavos

Después de simular la fase de grupos, se ordenan las tablas de cada grupo usando estos criterios:

1. puntos;
2. diferencia de goles;
3. goles a favor.

Luego clasifican:

- los 12 primeros de grupo;
- los 12 segundos de grupo;
- los 8 mejores terceros.

El sistema almacena los 32 clasificados en un arreglo dinámico de punteros:

```
Equipo** clasificadosR16;
```

5.9. Fases finales

Las fases finales se simulan de manera progresiva:

1. dieciseisavos: 32 equipos, 16 partidos;
2. octavos: 16 equipos, 8 partidos;
3. cuartos: 8 equipos, 4 partidos;
4. semifinales: 4 equipos, 2 partidos;
5. tercer puesto;
6. final.

En las fases finales no se permiten empates. Si ocurre uno, el partido se resuelve en prórroga y se suma un gol al equipo ganador, usando un criterio por ranking FIFA.

5.10. Estadísticas finales

Al finalizar el torneo se muestran:

- campeón;
- subcampeón;
- tercer puesto;
- cuarto puesto;
- máximo goleador del equipo campeón;
- tres mayores goleadores del torneo;
- equipo con más goles históricos actualizados.

6. Medición de recursos

Se implementó la clase MedidorRecursos. Esta clase permite contar iteraciones y estimar memoria.

6.1. Conteo de iteraciones

El contador de iteraciones se reinicia al comienzo de cada funcionalidad importante con:

```
medidor.reiniciar();
```

Dentro de los ciclos principales se usa:

```
medidor.sumarIteracion();
```

Al finalizar la funcionalidad se imprime el resumen:

```
mostrarRecursos("Nombre de la funcionalidad");
```

6.2. Estimación de memoria

La memoria se estima sumando:

- tamaño de la clase Torneo;
- arreglos dinámicos de equipos;
- cadenas dinámicas de cada equipo;
- plantillas de jugadores;
- cadenas dinámicas de jugadores;
- grupos;
- arreglos de punteros;
- partidos;
- clasificados;
- top 4.

Estos valores muestran un comportamiento coherente: la memoria aumenta cuando se crean estructuras nuevas, como jugadores, partidos y fases finales.

7. Eficiencia y manejo de memoria

7.1. Uso de punteros para evitar copias

Una decisión importante fue almacenar equipos mediante punteros en grupos, bombos, clasificados y top 4. Esto evita duplicar objetos completos.

Por ejemplo:

```
Equipo** clasificadosR16;
Equipo** top4;
Equipo** bombos[4];
```

Estas estructuras solo referencian objetos existentes en el arreglo principal:

```
Equipo* equipos;
```

Esto reduce memoria y mantiene la consistencia de las estadísticas. Si un equipo actualiza sus goles históricos después de un partido, todas las estructuras que apuntan a ese equipo ven el mismo dato actualizado.

7.2. Propiedad de memoria

Se definió claramente qué clase es dueña de cada recurso:

- Torneo es dueño del arreglo principal de equipos.
- Equipo es dueño de su plantilla de jugadores.
- Grupo solo referencia equipos, no los destruye.
- Partido solo referencia equipos, no los destruye.
- Partido sí destruye sus cadenas dinámicas, árbitros y convocados.

Esta separación evita dobles liberaciones y punteros inválidos.

En las clases con memoria dinámica se implementó lo siguiente:

- destructor;
- constructor de copia;
- operador de asignación.

7.3. Liberación de memoria

Los destructores liberan la memoria asignada con `new[]` usando `delete[]`. De forma conceptual:

```
Torneo::~Torneo() {
    delete[] equipos;
    delete[] grupos;
    delete[] partidosGrupos;
    delete[] clasificadosR16;
    delete[] partidosR16;
    delete[] partidosR8;
    delete[] partidosQF;
    delete[] partidosSF;
    delete[] partidosFinales;
    delete[] top4;

    for(int i = 0; i < 4; i++) {
        delete[] bombos[i];
    }
}
```

7.4. Relación entre memoria y ejecución

Durante la ejecución, la memoria inicial es baja porque solo existen los objetos principales. Después de cargar equipos aumenta por las cadenas dinámicas. Luego, al crear jugadores, el consumo crece de forma importante porque cada uno de los 48 equipos reserva una plantilla de 26 jugadores. Finalmente, al generar partidos y fases finales, se agregan los arreglos dinámicos de partidos.

7.5. Resultados observados

Durante la simulacion del codigo se verificó que:

- se cargaron correctamente 48 equipos;
- los equipos se mostraron con país, confederación y ranking correctos;
- se formaron 4 bombos de 12 equipos;
- se sortearon 12 grupos de 4 equipos;
- se generaron 72 partidos de fase de grupos;

- se clasificaron 32 equipos a dieciseisavos;
- se simularon todas las fases finales;
- se imprimió el top 4 del torneo;
- el medidor de recursos se mostró al finalizar cada funcionalidad.

7.6. Ejemplo de estadísticas finales

Una ejecución de prueba produjo una salida similar a la siguiente:

```
Top 4 del mundial:
1. Campeón: Tunisia
2. Subcampeón: Portugal
3. Tercer puesto: Morocco
4. Cuarto puesto: Japan

Máximo goleador del equipo campeón:
#1 nombre1 apellido1 | Goles: 2

Tres mayores goleadores de toda la copa:
1. Morocco | #12 nombre12 apellido12 | Goles: 4
2. Japan | #15 nombre15 apellido15 | Goles: 4
3. France | #12 nombre12 apellido12 | Goles: 3

Equipo con más goles históricos actualizados:
Japan | GF históricos: 42
```

Debido al uso de aleatoriedad, los resultados pueden cambiar en cada ejecución.

8. Problemas de desarrollo afrontados

8.1. Lectura incorrecta del CSV

Inicialmente el archivo CSV se estaba leyendo como si estuviera separado por comas. Sin embargo, el archivo real utilizaba punto y coma. Esto causaba que una línea completa fuera interpretada como un solo campo. La solución fue implementar manualmente una forma de leer y separar los datos usando el carácter `;` como separador y saltar las líneas de encabezado con la función `esNumeroCSV()`.

8.2. Sorteo de grupos

Al principio al realizar el sorteo de grupos el algoritmo fallaba porque a veces no encontraba una combinacion valida, por lo que, se implementó un algoritmo aleatorio de múltiples intentos para que siempre encuentre una combinación valida para sortear los grupos correctamente.

9. Simplificaciones realizadas

Se realizaron algunas simplificaciones permitidas por el enunciado o justificadas por el alcance:

- la sede de todos los partidos se dejó como `nombreSede`;
- la hora de todos los partidos se dejó como `00:00`;
- los árbitros se dejaron como códigos fijos;
- la fase final usa una fecha fija: 10/07/2026;

10. Conclusiones

El desarrollo de UdeAWorldCup permitió aplicar conceptos fundamentales de programación orientada a objetos en C++. Se modelaron entidades reales mediante clases, se usó memoria dinámica para representar equipos, jugadores, grupos, partidos y rondas, y se implementó un flujo completo de simulación del Mundial 2026.

La solución permite cargar datos desde archivo, crear plantillas, organizar bombos, sortear grupos, simular partidos, clasificar equipos, ejecutar fases finales y generar estadísticas del torneo. Además, se agregó un medidor de recursos que permite observar la relación entre las funcionalidades ejecutadas, las iteraciones estimadas y el consumo aproximado de memoria.

Uno de los aprendizajes principales fue la importancia de definir claramente la propiedad de la memoria. Algunas clases son dueñas de sus recursos y deben ser liberadas; otras solo almacenan punteros a objetos existentes. Esta diferencia fue clave para evitar fugas de memoria y dobles liberaciones.

También se evidenció que el diseño modular facilita la evolución del sistema, ya que cada clase pudo desarrollarse y probarse de manera independiente antes de integrarse en la clase principal Torneo.

En general, el proyecto cumple con el objetivo de representar un torneo mundial de fútbol mediante estructuras orientadas a objetos, control explícito de memoria y simulación automática de resultados.